

MATRIMORPHOSIS – 2026

ASSIGNMENT 4

INTRODUCTION TO SHELL PROGRAMMING

Name : Shahnaz Shafreen S
College : Bannari Amman Institute of Technology
Department : Computer Science and Engineering
Batch : 2023 - 2027
Email ID : shafreenshahnaz@gmail.com
Date : 04-09-2026

1. What are environment variables?

- Environment variables are named variables provided by the operating system and shell that store information used by programs and processes. They contain useful information such as the user's home directory, the command search path, the current username, and the shell being used. Some common environment variables are PATH, HOME, USER, and SHELL. These variables can be accessed by programs running from the shell and are inherited by child processes.
- Environment variables can be considered similar to global and local variables in terms of their availability, although their scope works differently from variables in programming languages. Some environment variables can be configured system-wide so that they are available to multiple users, while others can be set specifically for an individual user. System-wide settings can commonly be found in files such as /etc/environment and /etc/profile, whereas user-specific settings are commonly placed in files such as ~/.profile and ~/.bashrc.
- Command to view environment variables:
 - printenv
 - The env command can also be used to display environment variables.
- Command to view a system-wide configuration file:
 - cat /etc/environment
 - A user can set an environment variable using the export command.
- Command:
 - export MY_VARIABLE="Hello"
 - The value can then be checked using:
 - echo \$MY_VARIABLE

2. Write a shell program that prints the numbers from 1 to 100 with each line having the number in digits, followed by a space and then the number in words.

- A shell program can be used to print the numbers from 1 to 100 along with their corresponding names in words. The program uses a loop to go through each number from 1 to 100. For each number, the program first prints the number in digits and then prints its equivalent in words. The numbers from 1 to 19 can be stored directly, while the tens values such as 20, 30, 40, and so on can be handled separately.
- For numbers from 20 to 99, the program can determine the tens digit and the units digit separately. If the units digit is zero, only the tens name is printed. Otherwise, the tens name is followed by the corresponding unit's name. The number 100 is handled separately and is printed as 100 hundred.

- Shell program:

```
#!/bin/bash
ones=(zero one two three four five six seven eight nine ten eleven twelve
thirteen fourteen fifteen sixteen seventeen eighteen nineteen)
tens=( "" "" twenty thirty forty fifty sixty seventy eighty ninety )
for ((number=1; number<=100; number++))
do
    if [ $number -lt 20 ]
    then
        echo "$number ${ones[$number]}"
    elif [ $number -lt 100 ]
    then
        tens_digit=$((number / 10))
        ones_digit=$((number % 10))

        if [ $ones_digit -eq 0 ]
        then
            echo "$number ${tens[$tens_digit]}"
        else
```

```

        echo "$number ${tens[$tens_digit]} ${ones[$ones_digit]}"
    fi
else
    echo "$number hundred"
fi
done

```

➤ Expected output:

```

1 one
2 two
3 three
...
19 nineteen
20 twenty
21 twenty one
22 twenty two
...
98 ninety eight
99 ninety nine
100 hundred

```

3. How many unique processes are currently running on your Linux system?

- The `ps` command can be used to display information about the processes currently running on a Linux system. To find the number of unique processes, the command output can be combined with `sort`, `uniq`, and `wc`. The `-o cmd` option of `ps` displays the command associated with each process. Sorting the output and removing duplicate command names makes it possible to count the unique processes.
- Command using `ps`, `sort`, `uniq`, and `wc`:
 - `ps -e -o cmd | sort | uniq | wc -l`
 - The `ps -e` option lists processes running on the system, while `-o cmd` displays their command names. The `sort` command

arranges the output, `uniq` removes consecutive duplicate entries, and `wc -l` counts the remaining lines. The resulting number represents the number of unique command entries in the process list.

➤ Alternative method using `cut`

- The same task can also be approached using the `cut` command by extracting the relevant part of the process information before sorting and counting the unique entries.
- Command: `ps -e -o cmd | cut -d' ' -f1 | sort -u | wc -l`
- The `cut` command extracts the required field from each line, `sort -u` sorts the entries and removes duplicates, and `wc -l` counts the unique entries.

4. Write a shell script that takes 2 integers as command line arguments and prints their sum. Does your script work if one or both of the integers are negative numbers or zero? If not, modify your program or the way you run the script to make it work.

- A shell script can accept values entered after the script name as command line arguments. These arguments are accessed using positional parameters such as `$1` and `$2`, where `$1` represents the first argument and `$2` represents the second argument. The arithmetic expansion syntax `$(( ))` can then be used to calculate the sum of the two integers.
- The script works with positive numbers and zero without any special modification. Negative numbers can also be passed as command line arguments, but when a negative value is supplied directly, the shell may interpret it as an option in some commands. Therefore, using `--` before the negative arguments or placing the negative values in a suitable form can prevent them from being interpreted as options.
- Shell script:

```
#!/bin/bash
sum=$(( $1 + $2 ))
```

```
echo "Sum = $sum"
```

- Command to run the script with positive numbers:

```
./sum.sh 10 20
```

Expected output: Sum = 30

- Command with zero:

```
./sum.sh 10 0
```

Expected output: Sum = 10

- Command with negative numbers:

```
./sum.sh -10 20
```

Expected output: Sum = 10

- The script itself handles negative numbers and zero correctly because shell arithmetic supports signed integers. Thus, no change to the arithmetic expression is required; the arguments can simply be passed with their negative sign.

5. Explore the grep command. Can you find an alternative for wc -l, using grep?

- The grep command is used to search for lines that match a particular pattern in text. The wc -l command counts the number of lines in a file or command output. An alternative to wc -l can be created using grep by using a pattern that matches every non-empty line and then counting the matching lines.
- One simple method is to use grep -c, where the -c option counts the number of matching lines instead of displaying them. The pattern . matches any character, so it can be used to count lines containing at least one character.
- Command:
 - grep -c .
 - For example, if the output of another command is piped to grep, it can count the lines containing characters.
 - Example: cat file.txt | grep -c .
 - This provides an alternative to: wc -l file.txt
- However, grep -c . counts non-empty lines, whereas wc -l counts all lines, including empty lines. Therefore, they produce the same result only when the

input does not contain empty lines.

6. What commands (or combination of commands) can be used to pipe the output of the program in Question 2 so that only the lines containing 2-digit numbers (i.e., 10 to 99) are printed??

- The output of the shell program in Question 2 contains numbers from 1 to 100, with each number followed by its name in words. The `grep` command can be used to filter this output and display only the lines beginning with two-digit numbers. Since the required range is from 10 to 99, the pattern can be written to match numbers having exactly two digits at the beginning of each line.
- Command:
 - `./numbers.sh | grep '^[0-9][0-9] '`
 - Here, `^` indicates the beginning of a line, `[0-9]` matches a digit, and using it twice matches exactly two digits. The space after the second `[0-9]` ensures that the pattern matches the number followed by a space, as in the output of Question 2.
- Alternative command using `grep -E`:
 - `./numbers.sh | grep -E '^[0-9]{2} '`
 - The `-E` option enables extended regular expressions, and `{2}` specifies that exactly two digits should occur at the beginning of the line. Therefore, the output will contain only the lines for 10 through 99, while 1–9 and 100 will be excluded.